

CS440: Face and Digit Classification Project

Deepti Rao [dr1197]

Namulun Togochoh [nt476]

Introduction

Our report covers the implementation and comparison of three machine learning classifiers on two image classification tasks: handwritten digit recognition (0-9) and face detection (whether the art provided in the given files displays a face vs. no face). The three classifiers are an implementation of the Perceptron algorithm, a neural network built from scratch using NumPy, and a neural network built using PyTorch.

Both tasks use the same type of input: ASCII images where each character is either a background space or an ink character ('+' or '#'). We use raw pixel values as features (0 for background and 1 for ink) which are flattened into a single vector.

To evaluate how each classifier scaled with data, we trained each one on 10% through 100% of the available training data (increasing each trial by another 10% of the data used) and measured accuracy, prediction error, and training time on the test set. Each training fraction was repeated 5 times with different random subsamples, and the results were averaged to get a reliable estimate of each classifier's performance.

Quick Algorithm Recaps

Perceptron:

The first part of the assignment was to implement the Perceptron algorithm. The perceptron is the simplest of the three classifiers. It's a linear model that learns one weight vector per class. To classify an input image, it computes a score for each class by taking the dot product of the weight vector with the flattened image, then picks the class with the highest score. There are no hidden layers or anything of the sort, and the decision boundary is always a straight line.

For digit classification, we maintained 10 weight vectors, one per digit, each being size 784 (derived from the 28 x 28 image). For face detection, a single weight vector of size 4200 (derived from the 70 x 60 image) is used, and the classifier predicts 'face' if the score is greater than or equal to 0.

When the classifier makes a mistake, it adds the input to the weights of the correct class and subtracts it from the weights of the wrongly predicted class. This is repeated for 3 full epochs over the training data.

Neural Network [Scratch]:

The next part of the assignment was then to build a neural network, consisting of 4 layers total (one input layer, two hidden layers, and one output layer), from scratch using custom implementations of forward pass, back-propagation, and weight updating.

Forward pass is the first step required to train a model using a neural network. It is responsible for computing the model's output based on the current weights and biases. Going layer by layer, it first calculates the weighted sum of the current layer ($z = \text{dot product}(\text{current layer}, \text{weight matrix}) + \text{bias}$). Then, it applies the corresponding activation function (ReLU for hidden layers and either softmax or sigmoid for the output layer) to non-linearize the result, stored in value a . This result will then be used as the next input, and this procedure is repeated until the final output is calculated. All z -values, a -values, weights, and biases are stored for use in back-propagation.

After forward pass is completed, the neural network will use back-propagation to calculate the most optimal way to update the model's weights and biases to improve its performance (through minimization of the difference between the predicted and actual outputs).

It does this by calculating the gradient descent of the weights and biases, which is done by finding the derivatives of the z -values, a -values, weights, and biases for use in the chain-rule.

Finally, the model is updated by subtracting the product of the learning rate and the gradient of the weight/bias from each weight and bias in the neural network. This ultimately allows the model to “learn” and theoretically improve in its ability to predict outputs based on given data.

For digit classification, we flattened the labels to a matrix of size 10 (with each index being a submatrix representing the label). For face detection, we reshaped the labels to a matrix of size (1, N) containing only 0 or 1. In prediction, the model compares the computed and actual outputs for a total of 50 epochs over the training data.

Neural Network [PyTorch]:

The PyTorch neural network uses the same architecture as the scratch network, but is built using PyTorch's `nn.Module`. The main difference is that instead of manually computing the gradients, PyTorch handles backpropagation automatically. PyTorch is usually significantly faster than implementations done from scratch due to optimization of code and access to GPUs.

Results

The tables below show the mean accuracy, mean prediction error, standard deviation of error, and mean training time for each classifier at each training fraction. Each value is averaged over 5 independent random subsamples of the training data.

Digit Classification:

Perceptron:

Training %	Accuracy	Error	Std Dev	Train Time
10%	73.40%	26.60%	$\pm 1.35\%$	0.011 s
20%	74.42%	25.58%	$\pm 2.36\%$	0.020 s
30%	76.14%	23.86%	$\pm 2.18\%$	0.029 s
40%	78.54%	21.46%	$\pm 2.32\%$	0.037 s
50%	75.30%	24.70%	$\pm 1.85\%$	0.054 s
60%	78.96%	21.04%	$\pm 1.17\%$	0.061 s
70%	79.24%	20.76%	$\pm 2.77\%$	0.074 s
80%	79.50%	20.50%	$\pm 1.97\%$	0.083 s
90%	77.12%	22.88%	$\pm 1.37\%$	0.095 s
100%	80.26%	19.74%	$\pm 1.00\%$	0.104 s

Neural Network [Scratch]:

Training %	Accuracy	Error	Std Dev	Train Time
10%	82.02%	17.98%	$\pm 1.12\%$	0.898 s
20%	83.48%	16.52%	$\pm 0.99\%$	3.212 s

30%	84.42%	15.58%	$\pm 0.64\%$	4.178 s
40%	86.00%	14.00%	$\pm 0.71\%$	8.720 s
50%	86.60%	13.40%	$\pm 0.80\%$	9.553 s
60%	87.72%	12.28%	$\pm 0.83\%$	14.417 s
70%	88.18%	11.82%	$\pm 0.44\%$	13.555 s
80%	88.86%	11.14%	$\pm 0.36\%$	21.025 s
90%	89.10%	10.90%	$\pm 0.58\%$	23.735 s
100%	90.04%	9.96%	$\pm 0.52\%$	30.898 s

Neural Network w/ PyTorch:

Training %	Accuracy	Error	Std Dev	Train Time
10%	82.08%	17.92%	$\pm 1.44\%$	2.480 s
20%	84.02%	15.98%	$\pm 1.21\%$	6.420 s
30%	86.00%	14.00%	$\pm 0.78\%$	5.620 s
40%	86.50%	13.50%	$\pm 1.02\%$	11.540 s
50%	87.52%	12.48%	$\pm 0.79\%$	4.610 s
60%	88.26%	11.74%	$\pm 0.57\%$	5.760 s
70%	88.80%	11.20%	$\pm 0.44\%$	6.770 s
80%	89.42%	10.58%	$\pm 0.58\%$	7.360 s

90%	90.20%	9.80%	$\pm 0.34\%$	8.080 s
100%	90.48%	9.52%	$\pm 0.41\%$	8.680 s

Face Classification:

Perceptron:

Training %	Accuracy	Error	Std Dev	Train Time
10%	69.87%	30.13%	$\pm 4.07\%$	0.000 s
20%	79.47%	20.53%	$\pm 2.68\%$	0.001 s
30%	77.60%	22.40%	$\pm 4.84\%$	0.005 s
40%	81.73%	18.27%	$\pm 3.71\%$	0.004 s
50%	82.00%	18.00%	$\pm 2.86\%$	0.005 s
60%	84.27%	15.73%	$\pm 1.50\%$	0.009 s
70%	85.73%	14.27%	$\pm 1.91\%$	0.006 s
80%	85.33%	14.67%	$\pm 1.89\%$	0.007 s
90%	87.47%	12.53%	$\pm 1.86\%$	0.008 s
100%	85.73%	14.27%	$\pm 1.72\%$	0.009 s

Neural Network [Scratch]:

=== Scratch NN | Faces | 10% of training data ===

Mean training time: 1.106 s

Mean accuracy: 66.80%

Mean error: 33.20%

Std of error: 8.61%

=== Scratch NN | Faces | 20% of training data ===

Mean training time: 6.248 s

Mean accuracy: 79.60%

Mean error: 20.40%

Std of error: 3.34%

=== Scratch NN | Faces | 30% of training data ===

Mean training time: 4.126 s

Mean accuracy: 82.40%

Mean error: 17.60%

Std of error: 2.41%

=== Scratch NN | Faces | 40% of training data ===

Mean training time: 5.200 s

Mean accuracy: 86.00%

Mean error: 14.00%

Std of error: 3.60%

=== Scratch NN | Faces | 50% of training data ===

Mean training time: 5.930 s

Mean accuracy: 87.73%

Mean error: 12.27%

Std of error: 1.37%

=== Scratch NN | Faces | 60% of training data ===

Mean training time: 3.783 s

Mean accuracy: 89.07%

Mean error: 10.93%

Std of error: 1.50%

=== Scratch NN | Faces | 70% of training data ===

Mean training time: 6.235 s

Mean accuracy: 88.53%
 Mean error: 11.47%
 Std of error: 1.42%

=== Scratch NN | Faces | 80% of training data ===

Mean training time: 6.894 s
 Mean accuracy: 89.73%
 Mean error: 10.27%
 Std of error: 1.16%

=== Scratch NN | Faces | 90% of training data ===

Mean training time: 12.334 s
 Mean accuracy: 89.47%
 Mean error: 10.53%
 Std of error: 2.08%

=== Scratch NN | Faces | 100% of training data ===

Mean training time: 15.615 s
 Mean accuracy: 89.87%
 Mean error: 10.13%
 Std of error: 0.98%

Neural Network w/ PyTorch:

Training %	Accuracy	Error	Std Dev	Train Time
10%	66.67%	33.33%	± 6.31%	0.170 s
20%	82.40%	17.60%	± 1.96%	0.300 s
30%	84.67%	15.33%	± 4.36%	0.470 s
40%	86.80%	13.20%	± 1.15%	0.550 s
50%	87.33%	12.67%	± 1.46%	0.690 s

60%	90.13%	9.87%	$\pm 1.29\%$	0.830 s
70%	89.60%	10.40%	$\pm 0.90\%$	0.950 s
80%	89.87%	10.13%	$\pm 0.78\%$	1.110 s
90%	90.13%	9.87%	$\pm 0.78\%$	1.150 s
100%	90.80%	9.20%	$\pm 0.88\%$	1.360 s

For digit classification, the PyTorch network seemed to perform the best. At 100% training data, it reached 90.48% accuracy, compared to 80.26% for the perceptron implementation and 90.04% for the scratch network. The Adam optimizer and the small batch training gave it a meaningful advantage on the harder 10-class digit task.

For faces, all three classifiers were much closer to each other. At 100% training data, PyTorch reached 90.80%, the scratch network reached 89.87%, and the perceptron reached 85.73%. The face task is binary, which makes it easier for simpler models to do well. Even the perceptron performed reasonably, though it fell about 5 points behind the neural networks at full data.

Across all classifiers and both datasets, accuracy generally increased as more training data was used, but with diminishing returns. Most classifiers stopped improving significantly past around 60-70% of the data.

On digits, the PyTorch network improved the most smoothly, with the scratch network close behind. It went from 82.08% at 10% data to 90.48% at 100%, a steady climb with low variance throughout (you can see the progression on the tables above). The perceptron plateaued around 80% and showed more noise in the middle ranges.

On faces, all three classifiers improved more dramatically at the low end. Both of the neural networks seemed to jump from around 67% at 10% data to over 85% by 40%, which we think shows that the face task benefits a lot from even a moderate amount of training data. The perceptron also improved quickly but leveled off earlier and never quite caught up to the neural networks.

The perceptron was by far the fastest to train. On digits, it took under 0.11 seconds even at 100% of the training data, and on faces, it took under 0.01 seconds. This makes sense because it has no hidden layers, no backpropagation, and only does a single dot product per example.

The scratch network was the slowest overall. On faces, it reached over 12 seconds at 90% training data. This is because it runs 50 full-batch epochs, meaning it loops through all the training data 50 times and computes gradients over the entire dataset each time. The large input size for faces (4200 features) and larger hidden layers (128 units) also added to the cost.

The PyTorch network sat in between. On digits, it ranged from about 2.5 to 8.7 seconds, and on faces, it was very fast (under 1.4 seconds) because the face training set is small. Despite running 20 epochs, the mini-batch approach meant that each weight update was cheaper, and Adam converged faster than plain stochastic gradient descent, so fewer epochs were needed to reach good accuracy.

Conclusion

After implementing all 3 classifiers, the perceptron, the neural network from scratch, and the neural network using PyTorch, on digit and face classification tasks, we came up with the following takeaways:

1. PyTorch was the best performing classifier on digits by a pretty significant margin (90.48% vs around 80%), thanks to it being optimized as well as mini-batch training. For the face classification task, however, all of them came out to a similar accuracy.
2. The Perceptron algorithm was the fastest to train by a large margin (it was able to train in under a tenth of a second on the full dataset) while still achieving competitive accuracy on the face classification task.
3. The neural network built from scratch matched the Perception implementation on digit classification, and the neural network using PyTorch for face classification.
4. More training data consistently helped, but the most significant gains happened during the first 60-70% of the data, after which the improvements to the model were smaller.

Overall, if speed is the priority, the Perceptron algorithm would be a reasonable choice for binary tasks. If your ego is the priority, and you deem it absolutely important to take the time out of your day to implement everything from scratch when there are tools and libraries available to make the process much more optimized and efficient, building your own neural network from scratch would be your go-to option. However, if accuracy matters more for a task, and training time is not too much of a constraint, the PyTorch neural network is the best overall option.